

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D3 — Test Report

Part 2 Submission

22 May 2026

Halit Batuhan ASLAN

220104004003

Test Contents

6 invalid tests with codes and terminal outputs

3 sample correct tests with codes and terminal outputs

6 tests for interpreter checking

```
invalid_1.pyar X
invalid_1.pyar
1 // _____
2 // invalid_1.pyar
3 // DELIBERATE ERROR : Comma between analyze fields (old style, now illegal).
4 // The updated grammar requires each analyze field to end with ";" and uses
5 // NO separator between fields = a comma after the ";" is unexpected.
6 // EXPECTED ERROR : ParseError - expected RBRACE, received COMMA
7 //
8
9 dataset d = load("data/iris.csv");
10
11 model KNN k
12 {
13     k = 3
14 }
15
16 experiment e
17 {
18     train k on d.train;
19     collect metrics [accuracy];
20
21     analyze overfitting
22     {
23         threshold = 0.10;;
24         underfit_max_acc = 0.60;
25     }
26
27     compare by accuracy higher_is_better;
28     select best;
29 }
30
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

▼ TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python ".\run_tests.py" --file .\invalid_1.pyar

Parsing file: .\invalid_1.pyar

X ParseError:
[ParseError] at line 23, col 26
Expected : token type='RBRACE'
Received : token type='COMMA', value=';'

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Invalid Test 1 — Comma between analyze fields

Error: A comma is placed between fields inside the analyze block. In the current grammar, analyze fields are only terminated by ; with no separator between them. The parser raises an error when it encounters , where } is expected.

```
invalid_1.ayar  invalid_2.ayar X
invalid_2.ayar
1 //
2 // invalid_2.ayar
3 // DELIBERATE ERROR : Missing ";" at the end of the dataset declaration.
4 // EXPECTED ERROR : ParseError - expected SEMICOLON, received KEYWORD 'model'
5 //
6
7 dataset titanic = load("data/titanic.csv")
8 | | | | | | | | | | | | | | | | | | // <-- semicolon missing here
9
10 model KNN k
11 {
12     k = 3
13 }
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

> ▼ TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_2.ayar

Parsing file: .\invalid_2.ayar

X ParseError:
[ParseError] at line 10, col 1
Expected : token type='SEMICOLON'
Received : token type='KEYWORD', value='model'

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> █

Invalid Test 2 — Missing semicolon after dataset

Error: The ; is missing at the end of the dataset declaration. The parser expects a SEMICOLON at the end of the line but finds the model keyword on the next line instead.

```
invalid_3.ayr X
invalid_3.ayr
1 //
2 // invalid_3.ayr
3 // DELIBERATE ERROR : "RandomForest" is not a valid model type in ayr.
4 // Valid types: KNN, DecisionTree, SVM, NaiveBayes, LogisticRegression.
5 // EXPECTED ERROR : ParseError - expected a model type (...),
6 // received IDENTIFIER 'RandomForest'
7 //
8
9 dataset bank = load("data/bank.csv");
10
11 model RandomForest rf_model
12 {
13     n_estimators = 100,
14     max_depth = 10
15 }
16

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HM> python '.\run_tests.py' --file .\invalid_3.ayr

Parsing file: .\invalid_3.ayr

X ParseError:
[ParseError] at line 11, col 7
Expected : a model type (DecisionTree, KNN, LogisticRegression, NaiveBayes, SVM)
Received : token type='IDENTIFIER', value='RandomForest'
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HM>
```

Invalid Test 3 — Invalid model type (RandomForest) Error:
RandomForest is not a valid model type in Ayar. Only KNN, DecisionTree, SVM, NaiveBayes, and LogisticRegression are supported. The parser reads the unrecognized type as an IDENTIFIER and reports that a model type was expected.

```
invalid_4.ayr X
invalid_4.ayr
1 //
2 // invalid_4.ayr
3 // DELIBERATE ERROR : The opening "{" is on the same line as the model header.
4 // The layout rule requires every "{" and "}" to be on its own line.
5 // EXPECTED ERROR : LayoutError - "{" must be on its own line
6 //
7
8 dataset mnist = load("data/mnist.csv");
9
10 model SVM svm_model {
11     C = 1,
12     kernel = "poly"
13 }
14

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HM> python '.\run_tests.py' --file .\invalid_4.ayr

Parsing file: .\invalid_4.ayr

X LayoutError:
[LayoutError] at line 10, col 21
Rule : '{' and '}' must each be on their own line
Found : '{' on the same line as the preceding token
Preceding: token type='IDENTIFIER', value='svm_model' at line 10, col 11
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HM>
```

Invalid Test 4 — Layout violation (brace on same line) Error: The opening { is on the same line as the model header. Ayar's layout rule requires { and } to each appear on their own line. A LayoutError is raised.

```
invalid_5.ayr X
invalid_5.ayr
1 //
2 // invalid_5.ayr
3 // DELIBERATE ERROR : "pipeline" is not a valid top-level keyword.
4 // Valid top-level keywords: dataset, split, model, experiment, evaluate,
5 // report.
6 // EXPECTED ERROR : ParseError - expected a top-level declaration keyword,
7 // received IDENTIFIER 'pipeline'
8 //
9
10 dataset credit = load("data/credit_card.csv");
11
12 model LogisticRegression lr_credit
13 {
14     C = 1,
15     max_iter = 300,
16     solver = "saga"
17 }
18
19 pipeline credit_pipeline
20 {
21     train lr_credit on credit.train;
22     select best;
23 }
24

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HM> python '.\run_tests.py' --file .\invalid_5.ayr

Parsing file: .\invalid_5.ayr

X ParseError:
[ParseError] at line 19, col 1
Expected : a top-level declaration keyword ('dataset', 'split', 'model', 'experiment', 'evaluate', 'report')
Received : token type='IDENTIFIER', value='pipeline'
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HM>
```

Invalid Test 5 — Unknown top-level keyword (pipeline) Error: pipeline is not a recognized top-level keyword in Ayar. Valid top-level keywords are: dataset, split, model, experiment, evaluate, report. The parser raises an error at the unrecognized keyword.

```
invalid_6.ayear X
invalid_6.ayear
1 // sample_errors.ayear
2 //
3 // PURPOSE : Intentional errors - uncomment ONE block at a time.
4 // RUN      : python3 run_tests.py --file sample_errors.ayear
5 //
6
7 // — LAYOUT ERROR (active): '{' on same line as header —————
8 // Expected: LayoutError - '{' must be on its own line
9 report final_report {
10     metrics = [accuracy],
11     show = overfitting_analysis
12 }
13
14
15 // — GRAMMAR ERROR (comment out the layout error above first) ———
16 // Expected: ParseError - missing ';' after dataset
17 // dataset iris = load("iris.csv")
18 // model KNN k
19 // {
20 //     k = 3
21 // }
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_6.ayear

Parsing file: .\invalid_6.ayear

X LayoutError:
[LayoutError] at line 9, col 21
Rule : '{' and '}' must each be on their own line
Found : '{' on the same line as the preceding token
Preceding: token type='IDENTIFIER', value='final_report' at line 9, col 8

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Invalid Test 6 — Layout violation in report block Error: The { in the report block is placed on the same line as the header. This demonstrates the same layout rule as Invalid Test 4, applied to a different construct (report).

Valid Tests;

```
sample_5.ayear X
sample_5.ayear
1 // sample_minimal.ayear
2 //
3 // PURPOSE : Smallest possible valid program - good first smoke test.
4 // RUN      : python3 run_tests.py --file sample_minimal.ayear
5 // EXPECTED: Parser succeeds, AST has 3 top-level nodes.
6 // NOTE     : Every '{' and '}' is on its own line (layout rule).
7 //
8
9 dataset d = load("wine.csv");
10
11 model KNN m
12 {
13     k = 3
14 }
15
16 experiment e
17 {
18     train m on d.train;
19     evaluate m on d.validation;
20     collect metrics [accuracy];
21     compare by accuracy higher_is_better;
22     select best;
23 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\sample_5.ayear

Parsing file: .\sample_5.ayear

AST:
Program
DatasetDecl(name='d', path='wine.csv')
ModelDecl(type='KNN', name='m')
Field(key='k')
Literal(kind='int', value=3)
ExperimentDecl(name='e')
TrainStmt(model='m')
DatasetRef('d'.train)
EvaluateStmt(model='m')
DatasetRef('d'.validation)
CollectStmt(metrics=['accuracy'])
CompareStmt(metrics='accuracy', direction='higher_is_better')
SelectStmt(best)

✓ Parse succeeded.

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Valid Test — sample_5.ayear (Minimal Program) The smallest valid Ayar program. Contains a single dataset, one KNN model, and a basic experiment block. Verifies that all mandatory constructs (dataset, model, experiment, train, evaluate, collect, compare, select) work in their minimal form. The AST output produces 3 top-level nodes.

```
sample_minimal.ayr X
sample_minimal.ayr
1 // sample_minimal.ayr (D1 spec aligned)
2 //
3 // Smallest valid program.
4 // RUN: python3 run_tests.py --file sample_minimal.ayr
5 //
6
7 dataset d = load("wine.csv");
8
9 model KNN m
10 {
11   k = 3
12 }
13
14 experiment e
15 {
16   train m on d.train;
17   evaluate m on d.validation;
18   collect metrics [accuracy];
19   analyze overfitting
20   {
21     threshold = 0.05;
22   }
23   compare by accuracy higher_is_better;
24   select best;
25 }
26
27 evaluate comparison.best on d.test;
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> ✓ TERMINAL
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '._\run_tests.py' --file '._\sample_minimal.ayr'
Parsing file: '._\sample_minimal.ayr'

AST:
Program
  DatasetDecl(name='d', path='wine.csv')
  ModelDecl(type='KNN', name='m')
    Field(key='k')
      Literal(kind='int', value=3)
  ExperimentDecl(name='e')
    TrainStmt(model='m')
      DatasetRef('d','train')
    EvaluateStmt(model='m')
      DatasetRef('d','validation')
    CollectStmt(metrics=['accuracy'])
    AnalyzeStmt
      AnalyzeField('threshold' = 0.05)
    CompareStmt(metric='accuracy', direction='higher_is_better')
    SelectStmt(best)
  EvalStmt
    EvalTarget('comparison'.best)
    DatasetRef('d','test')

✓ Parse succeeded.
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>
```

Valid Test — sample_minimal.ayr (Extended Minimal) Extends the minimal program by adding an analyze overfitting block and a top-level evaluate statement. Tests the comma-free analyze field format and the dotted eval_target form (comparison.best).

```
sample_1.ayaz X
sample_1.ayaz
1 //
2 // sample_1.ayaz
3 // DESCRIPTION : Iris flower classification using KNN and DecisionTree.
4 // COVERS      : dataset decl, split, 2 model types (int/string/bool fields),
5 //              full experiment (all 7 statement types), dotted eval_target
6 //              (identifier.identifier form, no brackets), report.
7 //
8 // CRITICAL SYNTAX RULES (updated parser / D1 spec):
9 // analyze fields: each ends with ";" only – NO comma between fields
10 // eval_target: dotted form evaluate exp_name.best on dataset.role;
11 //
12
13 dataset iris = load("data/iris.csv");
14
15 split iris into
16   train(0.70),
17   validation(0.15),
18   test(0.15);
19
20 model KNN knn_clf
21 {
22   n_neighbors = 7,
23   weights = "distance",
24   leaf_size = 20
25 }
26
27 model DecisionTree dt_clf
28 {
29   max_depth = 8,
30   criterion = "gini",
31   min_samples_split = 4
32 }
33
34 experiment iris_exp
35 {
36   train knn_clf on iris.train;
37   train dt_clf on iris.train;
38
39   evaluate knn_clf on iris.validation;
40   evaluate dt_clf on iris.validation;
41
42   collect metrics [accuracy, precision, recall, f1];
43
44   analyze overfitting
45   {
46     threshold = 0.08;
47     underfit_max_acc = 0.55;
48   }
49
50   compare by f1 higher_is_better;
51
52   exclude if overfit or underfit;
53
54   select best;
55 }
56
57 // dotted eval_target: identifier.identifier (no brackets) – NEW form
58 evaluate iris_exp.best on iris.test;
59
60 report iris_report
61 {
62   metrics = [accuracy, precision, recall, f1],
63   show    = overfitting_analysis
64 }
65
```

Valid Test — sample_1.ayaz (Iris Classification) The most comprehensive test program. Uses two model types (KNN + DecisionTree), int/string/bool field values, all 7 experiment statement types (train, evaluate, collect, analyze, compare, exclude, select), the dotted eval_target form, and a report block. The AST output produces all node types and prints "Parse succeeded".

```
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\sample_1.ayar
```

```
Parsing file: .\sample_1.ayar
```

```
AST:
```

```
Program
  DatasetDecl(name='iris', path='data/iris.csv')
  SplitStmt(dataset='iris')
    SplitPart(role='train', ratio=0.7)
    SplitPart(role='validation', ratio=0.15)
    SplitPart(role='test', ratio=0.15)
  ModelDecl(type='KNN', name='knn_clf')
    Field(key='n_neighbors')
      Literal(kind='int', value=7)
    Field(key='weights')
      Literal(kind='string', value='distance')
    Field(key='leaf_size')
      Literal(kind='int', value=20)
  ModelDecl(type='DecisionTree', name='dt_clf')
    Field(key='max_depth')
      Literal(kind='int', value=8)
    Field(key='criterion')
      Literal(kind='string', value='gini')
    Field(key='min_samples_split')
      Literal(kind='int', value=4)
  ExperimentDecl(name='iris_exp')
    TrainStmt(model='knn_clf')
      DatasetRef('iris'.train)
    TrainStmt(model='dt_clf')
      DatasetRef('iris'.train)
    EvaluateStmt(model='knn_clf')
      DatasetRef('iris'.validation)
    EvaluateStmt(model='dt_clf')
      DatasetRef('iris'.validation)
    CollectStmt(metrics=['accuracy', 'precision', 'recall', 'f1'])
    AnalyzeStmt
      AnalyzeField('threshold' = 0.08)
      AnalyzeField('underfit_max_acc' = 0.55)
    CompareStmt(metric='f1', direction='higher_is_better')
    ExcludeStmt(conditions=['overfit', 'underfit'])
    SelectStmt(best)
  EvalStmt
    EvalTarget('iris_exp'.best)
    DatasetRef('iris'.test)
  ReportStmt(name='iris_report')
    ReportField(metrics=['accuracy', 'precision', 'recall', 'f1'])
    ReportField(show='overfitting_analysis')
```

```
✓ Parse succeeded.
```

```

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1> python .\ayar_interpreter.py .\sample_1.ayar
Running Ayar interpreter on '.\sample_1.ayar'...

=== FINAL EVALUATION: dt_clf on iris.test ===
Model : dt_clf (DecisionTree)
Split : iris.test
Metrics:
  accuracy : 0.9200
  f1        : 0.9000
  precision : 0.9200
  recall    : 0.8900

=====
=== REPORT: iris_report ===
=====
Best model : dt_clf (DecisionTree)
Metrics :
  accuracy : 0.9200
  precision : 0.9200
  recall    : 0.8900
  f1        : 0.9000
Overfitting analysis:
  knn_clf   : OK
  dt_clf    : OK ← best
%PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1>

```

This program exercises all seven experiment statement types across two model types — KNN and DecisionTree. The DecisionTree with `max_depth=8` wins by `f1` (0.90) over KNN with `k=5` (`f1=0.84`), demonstrating that the formula-based `simulate()` function produces meaningfully different outcomes for different hyperparameter values. Both models pass the overfitting check because the train-eval accuracy gap of 0.07 falls below the declared threshold of 0.10, so neither is excluded. The dotted `eval_target` form (`iris_exp.best`) is used for the top-level evaluate statement, and the report block shows the full overfitting analysis with all four metrics.

```

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1> python .\ayar_interpreter.py .\sample_2.ayar
Running Ayar interpreter on '.\sample_2.ayar'...

=== FINAL EVALUATION: svm_clf on spam.test ===
Model : svm_clf (SVM)
Split : spam.test
Metrics:
  f1        : 0.7500
  precision : 0.7800
  recall    : 0.7400

=====
=== REPORT: spam_report ===
=====
Best model : svm_clf (SVM)
Metrics :
  precision : 0.7800
  recall    : 0.7400
  f1        : 0.7500
Overfitting analysis:
  svm_clf   : OK ← best
  lr_clf    : OK
%PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1>

```

This program demonstrates three features not present in `sample_1`: the SVM and LogisticRegression model types, a collect metrics list that excludes accuracy ([precision, recall, f1] only), and a lower_is_better compare direction chosen to minimize false negatives in a spam detection context. SVM wins with recall 0.74 against LogisticRegression's 0.75 — the lower recall is preferred under lower_is_better, directly exercising the compare direction mechanism. The bracket `eval_target` form ([spam_exp.best]) is used for the top-level evaluate statement, which is syntactically distinct from the dotted form used in `sample_1`.


```

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1> python .\ayar_interpreter.py .\sample_3.ayar
Running Ayar interpreter on '.\sample_3.ayar'...

[Runtime Error – NoEligibleModelError]
'select best' was called but all models were excluded by the
preceding 'exclude if' statement.
Excluded models:
All models were excluded. No model remains in M.
To fix: relax the 'analyze overfitting' thresholds, or add
more models so that at least one survives the exclusion filter.

❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1>

```

This program intentionally triggers the NoEligibleModelError defined in D1 Rule P-SelectBest (failure case). The threshold is set to 0.05, which is deliberately stricter than the fixed train-eval gap of 0.07 produced by the interpreter's `_TRAIN_BONUS` constant. As a result, all five model types are tagged as overfit by P-AnalyzeOverfitting, the exclude if overfit statement empties M entirely, and select best raises the domain-specific runtime error rather than silently returning an overfit model. This demonstrates Ayar's loud-failure design: reliability (Sebesta §1.3.3) is prioritized over convenience.

```

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1> python .\2p4_train_only_select.ayar
Running Ayar interpreter on '.\2p4_train_only_select.ayar'...

=== FINAL EVALUATION: m on d.test ===
Model : m (KNN)
Split : d.test
Metrics:
  accuracy : 0.8000
    f1      : 0.7900
  precision : 0.8100
    recall  : 0.7800

❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1>

```

This program demonstrates that an experiment block can run with only a train statement and select best — no evaluate, no collect, no compare, no analyze. The interpreter selects the single trained model by default since there is nothing to compare or exclude. The final evaluation runs on d.test and produces the expected KNN metrics based on the formula with k=5. This exercises the minimal valid experiment path and confirms that none of the pipeline statements are syntactically mandatory.

```

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1> python .\2tc1_partial_split.ayar
Running Ayar interpreter on '.\2tc1_partial_split.ayar'...

[Ayar Interpreter] Type checking failed – execution aborted.
2 error(s) found:

[R2-UnsplitDataset] line 30: Dataset 'extra' was declared but never split. Add 'split extra into train(...), validation(...), test(...);' before referencing it.
[R2-UnsplitDataset] line 31: Dataset 'extra' was declared but never split. Add 'split extra into train(...), validation(...), test(...);' before referencing it.
❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1> python .\2tc4_compare_uncollected.ayar
Running Ayar interpreter on '.\2tc4_compare_uncollected.ayar'...

[Ayar Interpreter] Type checking failed – execution aborted.
1 error(s) found:

[R4-UncollectedMetric] line 21: 'compare by accuracy' references a metric that was not collected. Collected metrics: ['f1', 'precision', 'recall']. Add 'accuracy' to the 'collect metrics' list, or change the compare metric to one that is collected.
❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW\220104004003_D2_P1>

```

This program declares two datasets but only splits one of them. The type checker raises two R2-UnsplitDataset errors at lines 30 and 31, one for each reference to the unsplit dataset extra. The iris dataset references produce no errors since iris is correctly split. This demonstrates that R2 fires per-reference rather than per-dataset, and that valid and invalid references can coexist in the same program — only the invalid ones are reported.

This program uses compare by accuracy but the collect metrics list contains only [f1, precision, recall] — accuracy was never collected. The type checker raises one R4-UncollectedMetric error at line 21, reporting the exact metric name that was missing and listing which metrics were actually collected. Execution is aborted before any training occurs. This confirms that R4 correctly cross-references the compare statement against the collect statement within the same experiment block.